

dbt **dbt — data build tool**

TRANSFORMATION

ELT / SQL

Transformation SQL dans le DataWarehouse — Fiche de référence rapide

Contexte TP : dbt prend en charge la couche **T** de l'ELT. Les données brutes MySQL/TXT/Excel/JSON sont déjà chargées en PostgreSQL (staging). dbt construit les dimensions et la table de faits avec du SQL versionné et testé.

ETL VS ELT — QUELLE DIFFÉRENCE AVEC VOTRE PIPELINE ACTUEL ?

REPOSITIONNEMENT DE LA TRANSFORMATION

ETL (VOTRE PIPELINE PYTHON)

MySQL / TXT / Excel / JSON

→ Transform Python

→ Load PostgreSQL

VS

ELT AVEC DBT

MySQL / TXT / Excel / JSON

→ Load brut PostgreSQL

→ dbt Transform SQL

Avantage dbt : les transformations sont en SQL pur, versionnées dans Git, documentées et testées automatiquement. Pas de boîte noire Python.

STRUCTURE D'UN PROJET DBT

ARBORESCENCE TYPE

```
dwh_imprimantes/  
├─ models/  
│ └─ staging/ ← données brutes nettoyées  
│   ├── stg_ventes.sql  
│   └── stg_clients.sql  
└─ marts/ ← dimensions + faits  
    ├── dim_vendeur.sql  
    ├── dim_client.sql  
    └── fait_ventes.sql  
├─ tests/ ← tests SQL custom  
├─ macros/ ← fonctions Jinja réutilisables  
├─ dbt_project.yml ← config projet  
└─ profiles.yml ← connexion DB
```

MATÉRIALISATIONS

view

Vue SQL — recalculée à chaque requête. Légère, pour staging.

table

Table physique recréée à chaque run. Pour dimensions.

incremental

Ajoute seulement les nouvelles lignes. Pour faits volumineux.

ephemeral

CTE injecté — aucune table créée. Intermédiaire pur.

MODÈLES DBT — EXEMPLES TP IMPRIMANTES

STAGING/STG_VENTES.SQL

```

-- Nettoyage de la table brute MySQL
{{ config(materialized='view') }}

SELECT
    id_vente,
    id_vendeur,
    id_client,
    id_produit,
    -- Transformation : cast de date
    date_commande::DATE      AS date_commande,
    date_precommande::DATE   AS date_precommande,
    quantite,
    -- Valeur métier : flag précommande
    montant_vente::NUMERIC   AS montant_vente,
    CASE
        WHEN montant_precom > 0 THEN TRUE
        ELSE FALSE
    END                      AS est_precommande,
    km_parcourus,
    frais_voyage
FROM {{ source('raw', 'ventes') }}
WHERE montant_vente > 0     -- Filtre valeurs aberrantes

```

MARTS/DIM_VENDEUR.SQL

```

-- Dimension vendeur avec attributs enrichis
{{ config(
    materialized='table',
    schema='marts'
) }}

SELECT
    -- Clé surrogate générée par dbt
    {{ dbt_utils.generate_surrogate_key(['id_vendeur']) }}
        AS id_dim_vendeur,
    id_vendeur                      AS id_source,
    -- Normalisation noms
    UPPER(TRIM(nom))                AS nom,
    INITCAP(TRIM(prenom))           AS prenom,
    INITCAP(prenom) || ' ' || UPPER(nom)
                                    AS nom_complet,
    salaire::NUMERIC(10,2),
    date_embauche::DATE,
    ville_base,

```

```
-- Transformation : catégorisation ancienneté
CASE
    WHEN groupe = 'Junior' THEN 1
    WHEN groupe = 'Senior' THEN 2
    WHEN groupe = 'Expert' THEN 3
END AS rang_groupe
FROM {{ ref('stg_vendeurs') }}
```

MARTS/FAIT_VENTES.SQL — MODÈLE INCRÉMENTAL

```
{{ config(
    materialized='incremental',
    unique_key='id_vente',          -- Clé de déduplication
    incremental_strategy='merge'
) }}
```

```
SELECT
    v.id_vente,
    t.id_dim_temps,
    vd.id_dim_vendeur,
    c.id_dim_client,
    p.id_dim_produit,
    v.quantite,
    v.montant_vente,
    -- Mesure calculée : marge estimée (règle métier 60% de coût)
    v.montant_vente - (p.prix_unitaire * v.quantite * 0.60) AS marge_estimee,
    v.km_parcourus,
    v.frais_voyage,
    -- Rentabilité nette
    (v.montant_vente - (p.prix_unitaire * v.quantite * 0.60))
    - v.frais_voyage AS rentabilite_nette
FROM {{ ref('stg_ventes') }} v
JOIN {{ ref('dim_temps') }} t ON t.date_complete = v.date_commande
JOIN {{ ref('dim_vendeur') }} vd ON vd.id_source = v.id_vendeur
JOIN {{ ref('dim_client') }} c ON c.id_source = v.id_client
JOIN {{ ref('dim_produit') }} p ON p.id_dim_produit = v.id_produit

{% if is_incremental() %}
    -- En mode incrémental : seulement les nouvelles lignes
    WHERE v.date_commande > (SELECT MAX(date_commande) FROM {{ this }})
{% endif %}
```

TESTS DBT — SCHEMA.YML

```
version: 2
models:
  - name: dim_vendeur
    columns:
      - name: id_dim_vendeur
        tests:
          - unique          # Pas de doublons
          - not_null        # Jamais NULL
```

```

- name: groupe
  tests:
    - accepted_values:
        values: ['Junior', 'Senior', 'Expert']

- name: fait_ventes
  columns:
    - name: id_dim_vendeur
      tests:
        - relationships: # Intégrité référentielle
            to: ref('dim_vendeur')
            field: id_dim_vendeur

```

COMMANDES CLI ESSENTIELLES

<code>dbt init dwh_imprimantes</code>	Crée un nouveau projet dbt
<code>dbt debug</code>	Vérifie la connexion PostgreSQL
<code>dbt run</code>	Exécute tous les modèles
<code>dbt run --select dim_vendeur</code>	Exécute un modèle précis
<code>dbt run --select staging.*</code>	Exécute un dossier entier
<code>dbt test</code>	Exécute tous les tests
<code>dbt docs generate</code>	Génère la documentation
<code>dbt docs serve</code>	Ouvre la doc dans le navigateur
<code>dbt lineage</code>	Affiche le graphe de dépendances

FONCTIONS JINJA ESSENTIELLES DANS LES MODÈLES

```
{{ ref('dim_vendeur') }}
```

Référence un autre modèle dbt — crée la dépendance automatiquement

```
{{ source('raw', 'ventes') }}
```

Référence une table source brute déclarée dans sources.yml

```
{{ config(materialized='table') }}
```

Configure le modèle (matérialisation, schéma, tags...)

INTÉGRATION AIRFLOW + DBT — LE COMBO PARFAIT

```

# Dans votre DAG Airflow — remplacer PythonOperator transform par :
from airflow.operators.bash import BashOperator

dbt_run = BashOperator(
    task_id='dbt_transform',
    bash_command='cd /opt/dwh_imprimantes && dbt run --profiles-dir .',

```

```
)
```

```
dbt_test = BashOperator(  
    task_id='dbt_test',  
    bash_command='cd /opt/dwh_imprimantes && dbt test',  
)
```

Ordre : extraire → charger brut → dbt run → dbt test → rapport

```
extraire_sources >> charger_brut >> dbt_run >> dbt_test
```

Pattern recommandé : Airflow = orchestrateur (QUAND et dans quel ordre), dbt = transformateur (COMMENT les données sont structurées). Les deux sont complémentaires, pas redondants.